```
{View} = require './view'

class List extends View
  constructor: (@posts) ->
  render: ->
    all = (for post in @posts
      "<li><a href='#{post.slug}'>#{post.title}</a></li>"
    ).join ''
    @wrap """
    <ul>#{all}</ul>
    """

exports.List = List
```

Again, a view that renders a template stored in a variable.

Finally, the `Post` class appears in the next listing. It also inherits from the `View` class. This `Post` class is used to render the title and body of an individual post.

Listing 12.10. views/post.coffee

```
{View} = require './view'

class Post extends View
  constructor: (@post) ->
  render: ->
    @wrap """
    <h1>#{@post.title}</h1>
    <div class='content'>
    #{@post.body}
    </div>
    """

exports.Post = Post
```

Shorthand constructor that only sets a post property. The function body is empty.

Again, a view that renders a template stored in a variable.

That was a lot of listings. All of these modules are probably overkill for a site as simple as the one you're creating. That's deliberate. The preceding listings present a working Model-View-Controller (a technique for breaking a program into different parts). Whether or not you're familiar with Model-View-Controller isn't important here. What is important is that you've seen a reasonably sized application built from modules. By examining them, you've seen examples from a real program of how `require` and `exports` work.